

Natural Language Processing

100 Questions: Pre-processing, BoW, TF-IDF, Naïve Bayes, Evaluation

GARP RAI Exam Preparation

QUESTION: What is Natural Language Processing (NLP)?

OPTIONS:

- ❶ A technique for processing numerical data only
- ❷ An aspect of machine learning concerned with understanding and analyzing human language, both written and spoken
- ❸ A method for image recognition
- ❹ A database management technique

ANSWER: B

DETAILED TUTORIAL:

- **Definition:** NLP is the intersection of computer science, AI, and linguistics
- **Core purpose:** Enable computers to understand, interpret, and generate human language
- **Key challenge:** Most information exists in unstructured, free text format
- **Alternative names:** Content analysis, text mining, computational linguistics

EXAM TRAP: NLP is about **understanding human language** - not just processing text!

QUESTION: Why is NLP more challenging than modeling purely numerical data?

OPTIONS:

- ❶ Numerical data is always larger
- ❷ Textual data is often noisy with errors and ambiguities
- ❸ NLP requires more computing power
- ❹ Numerical data is more interesting

ANSWER: B

DETAILED TUTORIAL:

- **Key challenges in NLP:**
 - ❶ **Ambiguity:** Same words have different meanings (polysemy)
 - ❷ **Context dependence:** Meaning depends on surrounding text
 - ❸ **Noise:** Spelling errors, abbreviations, colloquialisms
 - ❹ **Informal language:** Social media, text messages
 - ❺ **Sarcasm and tone:** Hard for machines to detect
- **Examples:** "u r gr8" vs "you are great", "lol" in different contexts

EXAM TRAP: NLP challenges include **ambiguity, noise, and context** - not just technical issues!

Q3: NLP Applications in Finance - Tutorial

QUESTION: Which of the following is an NLP application in finance?

OPTIONS:

- ❶ Fraud detection using SEC filings
- ❷ Customer service call routing
- ❸ Sentiment analysis from social media
- ❹ All of the above

ANSWER: D

DETAILED TUTORIAL:

❶ Key NLP applications in finance:

- ❶ **Fraud detection:** SEC used NLP to detect accounting fraud
- ❷ **Call routing:** Keyword recognition for customer inquiries
- ❸ **Sentiment analysis:** Social media monitoring
- ❹ **Readability assessment:** Fog index measurement
- ❺ **Document similarity:** Assessing new information content

EXAM TRAP: NLP has [many financial applications](#) - fraud detection, sentiment, customer service!

QUESTION: What are the main benefits of NLP over manual document reading?

OPTIONS:

- ① NLP is always more accurate than humans
- ② Superior speed, no scope to miss aspects, and consistent application
- ③ NLP is cheaper than human labor
- ④ NLP doesn't make mistakes

ANSWER: B

DETAILED TUTORIAL:

- Key advantages of NLP:
 - ① **Speed:** Processes thousands of documents quickly
 - ② **Completeness:** No scope to miss any aspects
 - ③ **Consistency:** All documents considered identically

EXAM TRAP: NLP advantages = speed, completeness, and consistency!

QUESTION: What are the most common NLP data sources in finance?

OPTIONS:

- ① Only financial statements
- ② Corporate disclosures, social media posts, and news wires
- ③ Only social media
- ④ Only news articles

ANSWER: B

DETAILED TUTORIAL:

- **Primary data sources:**
 - ① **Corporate disclosures:** 10-Q, 10-K, prospectuses
 - ② **Social media:** X (Twitter), Reddit, Stocktwits, Seeking Alpha
 - ③ **News wires:** Wall Street Journal, Dow Jones newswire
- **Data collection:** API, web scraping with Python
- **Corpus:** Collection of all retrieved documents

EXAM TRAP: Common sources = corporate disclosures, social media, and news wires!

Q6: JPMorgan COiN Platform - Tutorial

QUESTION: What did JPMorgan Chase's COiN platform achieve?

OPTIONS:

- ❶ It made no significant impact
- ❷ Significantly reduced time required to review legal documents
- ❸ It replaced all human reviewers
- ❹ It only worked for English documents

ANSWER: B

DETAILED TUTORIAL:

• **COiN (Contract Intelligence) Platform:**

- Automated review of legal documents
- Focus on commercial loan agreements

• **Outcomes:**

- Significantly reduced review time
- Improved consistency in information extraction
- Improved accuracy in information extraction

EXAM TRAP: COiN reduced legal document review time significantly!

QUESTION: Which institutions use NLP for specific applications?

OPTIONS:

- ① Only JPMorgan Chase uses NLP
- ② HSBC (fraud detection), BoA (chatbots), BlackRock (sentiment analysis)
- ③ No financial institutions use NLP
- ④ Only the SEC uses NLP

ANSWER: B

DETAILED TUTORIAL:

- **NLP use cases in banking:**
 - ① **Fraud detection:** HSBC
 - ② **Customer service:** Bank of America (BoA) chatbots
 - ③ **Market sentiment:** BlackRock
- **Early example:** SEC used NLP to detect accounting fraud

EXAM TRAP: HSBC = fraud detection, BoA = chatbots, BlackRock = sentiment analysis!

QUESTION: What is a corpus in the context of NLP?

OPTIONS:

- ❶ A single document
- ❷ The collection of all documents retrieved for analysis
- ❸ A type of machine learning algorithm
- ❹ A data pre-processing technique

ANSWER: B

DETAILED TUTORIAL:

• **Definition:**

- From Latin "corpus" meaning "body"
- Collection of all documents available for analysis
- May include hundreds or thousands of documents

• **Characteristics:**

- All documents in the corpus are processed together
- Vocabulary built from the corpus

EXAM TRAP: Corpus = collection of all documents - not a single document!

QUESTION: Why is data pre-processing important in NLP?

OPTIONS:

- ❶ It's not important
- ❷ To convert raw text into a format suitable for analysis
- ❸ To make documents shorter
- ❹ To remove all words

ANSWER: B

DETAILED TUTORIAL:

- **Purpose of pre-processing:**
 - ❶ Convert text to machine-readable format
 - ❷ Remove non-textual information (HTML, tags)
 - ❸ Standardize the text
 - ❹ Reduce noise and improve accuracy
- **Initial cleaning:** Extract text from PDFs, remove HTML, handle emojis

EXAM TRAP: Pre-processing converts **raw text** to **analyzable format**!

QUESTION: What is a practical use of document similarity in finance?

OPTIONS:

- ❶ To make documents longer
- ❷ To assess whether a news article constitutes new information
- ❸ To delete duplicate documents
- ❹ To remove all documents

ANSWER: B

DETAILED TUTORIAL:

- Document similarity applications:
 - ❶ **New information assessment:** Determine if news article provides new information
 - ❷ **Disclosure tracking:** Assess if tone of accounting disclosures has changed

EXAM TRAP: Document similarity helps assess **new information content**!

QUESTION: What is tokenization in NLP pre-processing?

OPTIONS:

- ➊ Removing all punctuation from text
- ➋ Separating a document into sentences and then into words
- ➌ Converting all text to lowercase
- ➍ Removing stop words

ANSWER: B

DETAILED TUTORIAL:

- **Tokenization process:**
 - ➊ **Sentence tokenization:** Split at periods, question marks, exclamation marks
 - ➋ **Word tokenization:** Split sentences into words, remove punctuation, convert to lowercase

EXAM TRAP: Tokenization = **splitting into sentences and words!**

QUESTION: What are stop words and why are they removed?

OPTIONS:

- ❶ Important words that convey meaning
- ❷ Common words with no informational value (e.g., "the", "a", "also")
- ❸ All nouns and verbs
- ❹ All proper nouns

ANSWER: B

DETAILED TUTORIAL:

- **Stop words definition:**
 - Words with little informational value
 - Examples: "has", "a", "the", "also", "and"
 - Common in all documents
- **Why remove them:** Add noise, don't help distinguish documents, increase vector size
- **Caveat:** What constitutes a stop word varies by application

EXAM TRAP: Stop words = common words with no informational value!

QUESTION: What is stemming in NLP pre-processing?

OPTIONS:

- ❶ Removing all words from a document
- ❷ Replacing words with their core/root forms by cutting prefixes and suffixes
- ❸ Converting all text to uppercase
- ❹ Removing all punctuation

ANSWER: B

DETAILED TUTORIAL:

- **Stemming definition:**
 - Reduces words to their root forms
 - Cuts off prefixes and suffixes
 - Crude, rule-based approach
- **Examples:** "disappointing" → "disappoint", "running" → "run"
- **Limitation:** Can lose nuance, may create non-words

EXAM TRAP: Stemming = cutting prefixes/suffixes to root form!

Q14: Lemmatization Definition - Tutorial

QUESTION: What is lemmatization and how does it differ from stemming?

OPTIONS:

- ❶ Lemmatization is the same as stemming
- ❷ Lemmatization finds the dictionary form (lemma) considering word meaning and part of speech
- ❸ Lemmatization removes stop words
- ❹ Lemmatization converts text to lowercase

ANSWER: B

DETAILED TUTORIAL:

• **Lemmatization definition:**

- Finds the dictionary form (lemma) of words
- Considers word meaning and part of speech
- More sophisticated than stemming

• **Comparison:**

- Stemming: Rule-based, fast, can be inaccurate
- Lemmatization: Uses vocabulary, slower, more accurate

EXAM TRAP: Lemmatization = **dictionary form considering meaning and POS!**

QUESTION: What is Part-of-Speech (POS) tagging and why is it useful?

OPTIONS:

- ❶ Identifying all verbs in a document
- ❷ Labelling each word by its class (noun, verb, adjective, proper noun, etc.)
- ❸ Removing all adjectives from a document
- ❹ Counting the number of sentences

ANSWER: B

DETAILED TUTORIAL:

- **POS tagging definition:**
 - Labels each word with its grammatical category
 - Common tags: noun, verb, adjective, proper noun
- **Why it's useful:**
 - Handles ambiguity: "Reading" (town) vs "reading" (activity)
 - Handles case sensitivity: "Polish" (language) vs "polish" (to shine)

EXAM TRAP: POS tagging labels **each word by grammatical class!**

QUESTION: What problem can lowercasing cause in NLP?

OPTIONS:

- ❶ It makes text harder to read
- ❷ It can cause inaccuracies by removing case-based distinctions (proper nouns, different meanings)
- ❸ It doesn't affect NLP
- ❹ It makes words longer

ANSWER: B

DETAILED TUTORIAL:

- **Lowercasing problems:**
 - ❶ **Proper nouns:** "Reading" (town) vs "reading" (activity)
 - ❷ **Meaning loss:** "Polish" (language) vs "polish" (to shine)
- **Solution:** Apply POS tagging before lowercasing

EXAM TRAP: Lowercasing can lose **proper noun distinctions** - use POS tagging first!

QUESTION: What is a limitation of removing punctuation?

OPTIONS:

- ❶ It doesn't affect NLP
- ❷ It can cause loss of valuable context (questions, exclamations, sarcasm)
- ❸ It makes text shorter
- ❹ It removes all verbs

ANSWER: B

DETAILED TUTORIAL:

- **Punctuation problems:**
 - ❶ **Questions:** "Is the CEO performing well?" → may seem positive
 - ❷ **Exclamations:** May denote humor or sarcasm

EXAM TRAP: Removing punctuation loses **context** and **tone** information!

Q18: Stemming vs Lemmatization Usage - Tutorial

QUESTION: When might lemmatization be preferred over stemming?

OPTIONS:

- ① When speed is the only concern
- ② When accuracy and preserving meaning are important
- ③ When working with very short documents
- ④ When speed is critical

ANSWER: B

DETAILED TUTORIAL:

- **When to use lemmatization:**
 - Accuracy is critical (financial documents need precise analysis)
 - Meaning preservation (irregular verbs, context-specific forms)
- **When to use stemming:**
 - Speed is critical (very large documents, real-time processing)
- **Tradeoff:** Stemming = faster, less accurate; Lemmatization = slower, more accurate

EXAM TRAP: Lemmatization preferred for **accuracy**, stemming for **speed**!

QUESTION: How is the vocabulary for BoW determined?

OPTIONS:

- ① Only using pre-existing dictionaries
- ② Either imposed exogenously or created from every distinct word in the corpus
- ③ Only using the most common words
- ④ Only using the rarest words

ANSWER: B

DETAILED TUTORIAL:

- Vocabulary creation methods:
 - ① **Exogenous creation:** Created ahead of time, applied to documents
 - ② **Corpus-based creation:** Every distinct word in all documents

EXAM TRAP: Vocabulary = pre-defined OR from corpus words!

QUESTION: What is the Bag of Words (BoW) approach?

OPTIONS:

- ❶ Analyzing words in their original order
- ❷ Treating each word in a document independently, ignoring order
- ❸ Counting only the longest words
- ❹ Removing all words from documents

ANSWER: B

DETAILED TUTORIAL:

- **BoW definition:**
 - Treats each word distinctly and equally
 - Position and order are ignored
 - Counts occurrences in each document
- **Limitations:**
 - Loses context
 - Ignores negation words
 - Misses phrase meanings

EXAM TRAP: BoW ignores word order and position!

QUESTION: What is the difference between binary and count Bag of Words?

OPTIONS:

- ❶ Binary uses 1/0, count uses frequency counts
- ❷ Binary is more accurate than count
- ❸ Count ignores word frequencies
- ❹ There is no difference

ANSWER: A

DETAILED TUTORIAL:

- **Binary BoW:**
 - 1 if word appears, 0 if not
 - Presence/absence only
- **Count BoW:**
 - Counts how many times each word appears
 - Non-negative integers

EXAM TRAP: Binary = **presence/absence**, Count = **frequency counts**!

QUESTION: What is a Document Term Matrix (DTM)?

OPTIONS:

- ❶ A matrix of document titles
- ❷ A collection of document vectors with rows as documents and columns as terms
- ❸ A matrix of document dates
- ❹ A matrix of document authors

ANSWER: B

DETAILED TUTORIAL:

- **DTM definition:**
 - Rows = documents
 - Columns = terms/words
 - Entries = word counts or frequencies
- **Characteristics:**
 - Sparse (many zeros)
 - Large (many columns)

EXAM TRAP: DTM = documents as rows, terms as columns!

QUESTION: Why are document word vectors described as sparse?

OPTIONS:

- ① They contain many non-zero values
- ② They contain many zeros
- ③ They are all the same length
- ④ They are very small

ANSWER: B

DETAILED TUTORIAL:

- **Sparsity definition:**
 - Most words don't appear in each document
 - Vector entries are mostly zero
- **Problems:**
 - Computational inefficiency
 - Curse of dimensionality

EXAM TRAP: Sparse = **many zeros** - makes modeling inefficient!

QUESTION: Why is vector normalization needed in NLP?

OPTIONS:

- ❶ To make vectors longer
- ❷ To prevent repeated words from skewing analysis
- ❸ To remove all words
- ❹ To make vectors shorter

ANSWER: B

DETAILED TUTORIAL:

- **Why normalize:**
 - Repeated words can skew analysis
 - Similar to outliers in econometrics
- **L2 normalization:**
 - Divide by the L2 norm
 - Scales elements between 0 and 1
 - All documents become unit length

EXAM TRAP: L2 normalization = **scale to unit length!**

QUESTION: What is the formula for the L2 norm of a vector?

OPTIONS:

- ❶ $\|X\|_2 = \sum x_i$
- ❷ $\|X\|_2 = \sqrt{\sum x_i^2}$
- ❸ $\|X\|_2 = \sum x_i^2$
- ❹ $\|X\|_2 = \max(x_i)$

ANSWER: B

DETAILED TUTORIAL:

- **L2 norm formula:**
 - $\|X\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$
 - Also called Euclidean norm
- **Application in NLP:**
 - Normalize document vectors
 - Each vector becomes unit length

EXAM TRAP: L2 norm = square root of sum of squared elements!

QUESTION: What is the dictionary approach in NLP?

OPTIONS:

- ❶ A machine learning algorithm
- ❷ A pre-defined list of words sharing a common characteristic used for text analysis
- ❸ A method for removing stop words
- ❹ A way to tokenize documents

ANSWER: B

DETAILED TUTORIAL:

• **Dictionary approach definition:**

- Pre-defined list of words
- Words share a common characteristic
- Used to assess sentiment or topic

• **Common application:**

- Sentiment analysis
- Net sentiment = proportion positive - proportion negative

EXAM TRAP: Dictionary = pre-defined word lists with common characteristics!

QUESTION: In the newswire example, how is sentiment determined?

OPTIONS:

- ① Count only positive words
- ② Count only negative words
- ③ Count positive and negative words, subtract negatives from positives
- ④ Count all words equally

ANSWER: C

DETAILED TUTORIAL:

- **Sentiment calculation:**
 - Count positive words: rise, grow, resolve, relief
 - Count negative words: disappoint, worry, concern, decline
 - Net sentiment = positive count - negative count
- **Challenges:**
 - Counterfactual sentences
 - Complex sentence structure
 - Negation words can reverse meaning

EXAM TRAP: Net sentiment = positive count - negative count!

QUESTION: What is the Loughran-McDonald dictionary known for?

OPTIONS:

- ① Being the largest dictionary
- ② Being specifically designed for financial text analysis
- ③ Being the oldest dictionary
- ④ Only containing positive words

ANSWER: B

DETAILED TUTORIAL:

- **Loughran-McDonald dictionary:**
 - Developed for financial text analysis
 - Based on extensive examination of 10-K filings
 - Categories: positive, negative, uncertain, litigious, strong modal, weak modal
- **Accuracy:** Around 75% on Thomson Reuters news articles

EXAM TRAP: Loughran-McDonald = financial text dictionary - 75% accuracy!

QUESTION: What are the primary advantages of the dictionary approach?

OPTIONS:

- ❶ It's always 100% accurate
- ❷ Transparency, ease and speed of implementation
- ❸ It works for all types of text
- ❹ It requires no pre-processing

ANSWER: B

DETAILED TUTORIAL:

• Key advantages:

- ❶ **Transparency:** Word lists are explicit and clear
- ❷ **Ease of implementation:** Simple word counting
- ❸ **Comparability:** Same word lists across corpora

EXAM TRAP: Advantages = transparency, ease, speed, comparability!

QUESTION: What is a key limitation of the dictionary approach?

OPTIONS:

- ❶ It's too fast
- ❷ Dictionaries are only as good as the word lists they are built on
- ❸ It always overestimates sentiment
- ❹ It always underestimates sentiment

ANSWER: B

DETAILED TUTORIAL:

• **Key limitations:**

- ❶ **Quality dependence:** Only as good as word lists
- ❷ **Context limitation:** Designed for formal documents
- ❸ **Narrow scope:** Limited subjects available
- ❹ **Negation issues:** "not good" misclassified

EXAM TRAP: Dictionary = **only as good as word lists** - has many limitations!

QUESTION: What is an n-gram in NLP?

OPTIONS:

- ❶ A single word
- ❷ A sequence of n contiguous words treated as a single unit
- ❸ A type of machine learning algorithm
- ❹ A pre-processing technique

ANSWER: B

DETAILED TUTORIAL:

- **n-gram definition:**
 - n = 1: uni-gram (single word)
 - n = 2: bi-gram (two words)
 - n = 3: tri-gram (three words)
- **Example:** "credit spreads narrowed today" → bi-grams: ["credit spreads", "spreads narrowed", "narrowed today"]
- **Purpose:** Capture phrase meanings, handle negation

EXAM TRAP: n-gram = n contiguous words as one unit!

QUESTION: Why are n-grams needed beyond basic BoW?

OPTIONS:

- ❶ To make analysis faster
- ❷ Because some words have specific meaning when placed together
- ❸ Because basic BoW is too accurate
- ❹ To count words more precisely

ANSWER: B

DETAILED TUTORIAL:

• **Limitations of basic BoW:**

- ❶ **Loses phrase meaning:** "red herring" $\neq$ red + herring
- ❷ **Negation words:** "not good" $\neq$ not + good

• **How n-grams help:**

- Preserve phrase meaning
- Handle negations correctly
- Capture context

EXAM TRAP: n-grams capture **phrase meaning and negation context!**

QUESTION: How can n-grams help with negation words?

OPTIONS:

- ❶ By removing all negation words
- ❷ By treating "not" + following word as a single unit
- ❸ By ignoring negations
- ❹ By making negations positive

ANSWER: B

DETAILED TUTORIAL:

- **Negation problem:**
 - "not good" → should be negative
 - Basic BoW sees "not" and "good" separately
- **n-gram solution:**
 - Bi-grams capture "not good" as one unit
 - Preserves negation meaning

EXAM TRAP: n-grams pair **negation** + **following word** as one unit!

QUESTION: What does TF-IDF stand for and what does it measure?

OPTIONS:

- ① Term Frequency - Inverse Document Frequency; weights words by importance
- ② Total Frequency - Integrated Document Frequency
- ③ Text Frequency - Inverse Data Frequency
- ④ Term Frequency - Inverse Data Frequency

ANSWER: A

DETAILED TUTORIAL:

- **TF-IDF definition:**
 - **TF (Term Frequency):** How often word appears in document
 - **IDF (Inverse Document Frequency):** How rare the word is across documents
 - **TF-IDF:** $TF \times IDF$ = word importance score

EXAM TRAP: TF-IDF = Term Frequency $\times$ Inverse Document Frequency!

QUESTION: What is the formula for Term Frequency (TF)?

OPTIONS:

- ① $TF_{i,j} = w_{i,j} \times |v_j|$
- ② $TF_{i,j} = w_{i,j} / |v_j|$
- ③ $TF_{i,j} = |v_j| / w_{i,j}$
- ④ $TF_{i,j} = \log(w_{i,j})$

ANSWER: B

DETAILED TUTORIAL:

- **TF Formula:**
 - $TF_{i,j} = \frac{w_{i,j}}{|v_j|}$
 - $w_{i,j}$: number of times term i appears in document j
 - $|v_j|$: total number of terms in document j

EXAM TRAP: TF = word count / total words in document!

QUESTION: What is the formula for Inverse Document Frequency (IDF)?

OPTIONS:

- ① $IDF_i = \ln(D / \sum d_{i,j})$
- ② $IDF_i = D / \sum d_{i,j}$
- ③ $IDF_i = \sum d_{i,j} / D$
- ④ $IDF_i = \ln(\sum d_{i,j} / D)$

ANSWER: A

DETAILED TUTORIAL:

• **IDF Formula:**

- $IDF_i = \ln(D / \sum_j d_{i,j})$
- D = total number of documents
- $\sum d_{i,j}$ = number of documents containing term i

• **Interpretation:**

- Rare words: High IDF
- Common words: Low IDF
- Words in all documents: $IDF = 0$

EXAM TRAP: $IDF = \ln(\text{total documents} / \text{documents containing word})!$

QUESTION: Why do some words have zero TF-IDF values?

OPTIONS:

- ① Because they are not in the vocabulary
- ② Because they appear in every document (no discriminatory power)
- ③ Because they are too long
- ④ Because they are too short

ANSWER: B

DETAILED TUTORIAL:

• **Zero IDF explanation:**

- $IDF = \ln(D / \sum d_{i,j})$
- If word appears in all documents: $\sum d_{i,j} = D$
- $IDF = \ln(D/D) = \ln(1) = 0$

EXAM TRAP: Zero TF-IDF = word appears in all documents - no discriminatory power!

QUESTION: Why does the same word have different TF-IDF scores across documents?

OPTIONS:

- ① IDF varies by document
- ② TF varies by document (due to different lengths and frequencies)
- ③ TF-IDF is always constant
- ④ Vocabulary changes

ANSWER: B

DETAILED TUTORIAL:

- **Why TF varies:**
 - ① **Document length:** Longer documents have smaller TFs
 - ② **Frequency:** Word may appear multiple times
 - **IDF is constant:** Depends only on corpus
- EXAM TRAP:** IDF is constant across documents; TF varies $\Rightarrow$ TF-IDF varies!

QUESTION: What are common variations in TF-IDF implementation?

OPTIONS:

- ① Only one formula exists
- ② Variations include binary TF, log normalization, and different IDF formulas
- ③ TF-IDF is always the same
- ④ TF-IDF cannot be modified

ANSWER: B

DETAILED TUTORIAL:

- **Common variations:**
 - ① **TF variations:** Binary TF, log normalization, raw count
 - ② **IDF variations:** Smooth IDF, probabilistic IDF

EXAM TRAP: TF-IDF has many implementation variations - be aware!

QUESTION: In the TF-IDF example, which word has the highest score and why?

OPTIONS:

- 1 "this" - because it appears in all documents
- 2 "fed" - because it appears in only one short document
- 3 "down" - because it appears frequently
- 4 "stock" - because it appears in two documents

ANSWER: B

DETAILED TUTORIAL:

- **Highest scores:** "fed" = 0.28, "hikes" = 0.28, "rates" = 0.28
- **Why they're highest:**
 - Appear in only 1 out of 4 documents
 - $IDF = \ln(4/1) = 1.39$ (high)
 - Document length = 5 (short)
 - $TF = 1/5 = 0.20$
 - $TF-IDF = 0.20 \times 1.39 = 0.28$

EXAM TRAP: Rare words in short documents = **highest TF-IDF!**

QUESTION: What is the Naïve Bayes classifier?

OPTIONS:

- ❶ A regression algorithm
- ❷ A classification algorithm based on Bayes' theorem with independence assumption
- ❸ A clustering algorithm
- ❹ A dimensionality reduction technique

ANSWER: B

DETAILED TUTORIAL:

- **Naïve Bayes definition:**
 - Based on Bayes' theorem
 - Assumes features are independent (naïve assumption)
 - Used for text classification
- **Why "naïve":** Assumes word independence (unrealistic but works)

EXAM TRAP: Naïve = independence assumption - words treated independently!

QUESTION: What is Bayes' theorem in the context of classification?

OPTIONS:

- ① $P(c|d) = P(d|c)P(c)/P(d)$
- ② $P(c|d) = P(d|c) + P(c)$
- ③ $P(c|d) = P(c)/P(d)$
- ④ $P(c|d) = P(d)/P(c)$

ANSWER: A

DETAILED TUTORIAL:

● **Bayes' Theorem:**

- $P(c|d) = \frac{P(d|c) \times P(c)}{P(d)}$
- $P(c|d)$: Posterior (probability of class given document)
- $P(d|c)$: Likelihood (probability of document given class)
- $P(c)$: Prior (unconditional probability of class)

EXAM TRAP: Bayes = posterior = likelihood $\times$ prior / evidence!

QUESTION: What is the independence assumption in Naïve Bayes?

OPTIONS:

- ❶ Documents are independent of each other
- ❷ Each word in a document is independent of all other words
- ❸ Classes are independent of documents
- ❹ All documents have the same length

ANSWER: B

DETAILED TUTORIAL:

- **Independence assumption:**
 - $P(w_1, w_2, \dots, w_n|c) = P(w_1|c) \times P(w_2|c) \times \dots$
 - Words are conditionally independent given class

EXAM TRAP: Independence = words treated as independent - unrealistic but works!

QUESTION: How is prior probability $P(c)$ calculated in Naïve Bayes?

OPTIONS:

- ❶ $P(c) = n(c)/N$ (documents in class / total documents)
- ❷ $P(c) = N/n(c)$
- ❸ $P(c) = n(c) \times N$
- ❹ $P(c) = 1/N$

ANSWER: A

DETAILED TUTORIAL:

- **Prior probability formula:**
 - $P(c_i) = \frac{n(c_i)}{N}$
 - $n(c_i)$: number of documents labeled as class i
 - N : total number of documents

EXAM TRAP: Prior = class count / total documents!

QUESTION: How is the likelihood $P(w|c)$ calculated in Naïve Bayes?

OPTIONS:

- ❶ Count of word occurrences in class / total words in class
- ❷ Count of documents in class / total documents
- ❸ Word count / total words in corpus
- ❹ Always 0.5

ANSWER: A

DETAILED TUTORIAL:

• **Likelihood calculation:**

- $P(w_i|c_j) = \frac{\text{count of } w_i \text{ in class } c_j}{\text{total words in class } c_j}$
- Estimated from labeled training data

EXAM TRAP: Likelihood = word count in class / total words in class!

QUESTION: What is the posterior probability in Naïve Bayes?

OPTIONS:

- ➊ Probability of a document given its class
- ➋ Probability of a class given the document
- ➌ Probability of a word given its class
- ➍ The unconditional class probability

ANSWER: B

DETAILED TUTORIAL:

- **Posterior definition:**
 - $P(c|d)$
 - Probability that document d belongs to class c
 - Updated after seeing the document

EXAM TRAP: Posterior = probability of class given document!

QUESTION: What is Maximum A Posteriori (MAP) classification?

OPTIONS:

- ➊ Assign document to the class with the highest posterior probability
- ➋ Assign document to the class with the highest prior probability
- ➌ Assign document to the class with the highest likelihood
- ➍ Random classification

ANSWER: A

DETAILED TUTORIAL:

- **MAP definition:**
 - $\hat{c} = \arg \max_{c \in C} P(c|d)$
 - Choose class that maximizes posterior

EXAM TRAP: MAP = class with highest posterior probability!

QUESTION: What is the zero probability problem in Naïve Bayes?

OPTIONS:

- ➊ When a word appears in all documents
- ➋ When a word does not appear in a class in the training data, making the class probability zero
- ➌ When all words have zero frequency
- ➍ When no documents exist

ANSWER: B

DETAILED TUTORIAL:

- **Zero probability problem:**
 - Word doesn't appear in training data for a class
 - $P(w|c) = 0$
 - Product becomes zero: $\prod P(w_i|c) = 0$

EXAM TRAP: Zero probability = **word missing in training class** - need smoothing!

QUESTION: What is Laplace smoothing in Naïve Bayes?

OPTIONS:

- ① A method to remove stop words
- ② Adding 1 to all counts to avoid zero probabilities
- ③ A technique to stem words
- ④ A method to normalize vectors

ANSWER: B

DETAILED TUTORIAL:

- Laplace smoothing:
 - Adds 1 to all counts
 - Ensures all probabilities are non-zero
 - $P(w|c) = \frac{\text{count}(w,c)+1}{\text{total words in } c+|V|}$

EXAM TRAP: Laplace smoothing = add 1 to all counts to avoid zeros!

QUESTION: How does smoothing affect the estimated probabilities?

OPTIONS:

- 1 It makes probabilities more extreme
- 2 It pushes probabilities toward a uniform distribution
- 3 It has no effect on probabilities
- 4 It makes probabilities negative

ANSWER: B

DETAILED TUTORIAL:

- **Effect of smoothing:**
 - Prevents zero probabilities
 - Shrinks probabilities toward uniform
 - Similar to regularization (Lasso/Ridge)

EXAM TRAP: Smoothing = **shrinks toward uniform distribution** - similar to regularization!

QUESTION: What is the underflow problem in Naïve Bayes?

OPTIONS:

- ❶ When probabilities become too large
- ❷ When multiplying many small probabilities results in numbers too small for computers to handle
- ❸ When documents are too long
- ❹ When vocabulary is too small

ANSWER: B

DETAILED TUTORIAL:

• **Underflow definition:**

- Multiplying many probabilities < 1
- Product becomes extremely small
- Below computer precision

• **Solution:**

- Take logarithms
- $\log(P_1 \times \dots) = \sum \log(P_i)$

EXAM TRAP: Underflow = multiplying small probabilities - use logarithms!

QUESTION: Why is log transformation used in Naïve Bayes?

OPTIONS:

- ① To make probabilities larger
- ② To convert products to sums and avoid underflow
- ③ To remove stop words
- ④ To normalize the data

ANSWER: B

DETAILED TUTORIAL:

- Log transformation benefits:

- ① **Products to sums:** $\log(P_1 \times P_2) = \log(P_1) + \log(P_2)$
- ② **Preserves ordering:** Class with highest probability = highest log probability

EXAM TRAP: Log = products to sums - avoids underflow!

QUESTION: How are words in test data that were not in training data handled?

OPTIONS:

- ➊ They are added to the vocabulary
- ➋ They are ignored because they cannot help classify the document
- ➌ They cause the classification to fail
- ➍ They are treated as stop words

ANSWER: B

DETAILED TUTORIAL:

- **Unknown word handling:**
 - Words not in training vocabulary
 - No information about their distribution
 - Must be removed from test document

EXAM TRAP: Unknown words = **ignored** - they provide no classification information!

QUESTION: What are the two key assumptions of Naïve Bayes for document classification?

OPTIONS:

- ❶ Words are dependent and have different weights
- ❷ Words are independent and have equal weighting
- ❸ Words are ordered and have different weights
- ❹ Words are random and have no weights

ANSWER: B

DETAILED TUTORIAL:

- **Assumption 1: Independence**
 - Each word is independent of others
 - $P(w_1, w_2, \dots, w_n|c) = \prod P(w_i|c)$
- **Assumption 2: Equal weighting**
 - Each word has equal information value

EXAM TRAP: Naïve Bayes = independence + equal weighting!

QUESTION: What makes Naïve Bayes a generative classifier?

OPTIONS:

- ❶ It can generate new documents
- ❷ It learns the joint distribution $P(d, c)$ and can create new classifications
- ❸ It only learns decision boundaries
- ❹ It doesn't use probabilities

ANSWER: B

DETAILED TUTORIAL:

- **Generative classifiers:**
 - Model the joint distribution $P(d, c)$
 - Can generate new examples
 - Examples: Naïve Bayes

EXAM TRAP: Naïve Bayes = **generative** - learns joint distribution!

QUESTION: In the customer service example, what classification is made?

OPTIONS:

- 1 "Good" because it has the highest prior
- 2 "Bad" because it has the highest posterior probability
- 3 "Indifferent" because it's the middle class
- 4 Cannot be classified

ANSWER: B

DETAILED TUTORIAL:

- **Data:**
 - 4 good, 2 bad, 2 indifferent (8 total)
 - New document: words 1 and 2 present, words 3 and 4 absent
- **Calculated probabilities:**
 - $P(\text{good}|\text{words}) = 0$ (without smoothing)
 - $P(\text{bad}|\text{words}) = 0.67$ (highest)
 - $P(\text{indifferent}|\text{words}) = 0.33$

EXAM TRAP: Classification = **highest posterior probability** - not just prior!

QUESTION: How did Laplace smoothing change the classification?

OPTIONS:

- ❶ It didn't change the classification
- ❷ It changed "Good" from 0 to 2.9% probability
- ❸ It made "Bad" the lowest probability
- ❹ It made classification impossible

ANSWER: B

DETAILED TUTORIAL:

- **Before smoothing:**
 - $P(\text{good}|\text{words}) = 0$
 - $P(\text{bad}|\text{words}) = 0.67$
 - $P(\text{indifferent}|\text{words}) = 0.33$
- **After smoothing ($\lambda = 1$):**
 - $P(\text{good}|\text{words}) = 0.029$ (increased from 0)
 - $P(\text{bad}|\text{words}) = 0.583$ (decreased)
 - $P(\text{indifferent}|\text{words}) = 0.388$ (increased slightly)

EXAM TRAP: Smoothing eliminates zero probabilities but may not change classification!

QUESTION: How does Naïve Bayes handle more than two classes?

OPTIONS:

- ❶ It cannot handle more than two classes
- ❷ It computes posterior probability for each class and selects the highest
- ❸ It only uses binary classification
- ❹ It combines classes

ANSWER: B

DETAILED TUTORIAL:

- **Multi-class extension:**
 - Natural extension of binary case
 - Compute $P(c|d)$ for each class
 - Select class with highest probability

EXAM TRAP: Multi-class = choose class with highest posterior!

QUESTION: What are the main advantages of Naïve Bayes?

OPTIONS:

- ❶ It always has 100% accuracy
- ❷ Simplicity, speed, and solid classification accuracy
- ❸ It works without any assumptions
- ❹ It requires no training data

ANSWER: B

DETAILED TUTORIAL:

• Key advantages:

- ❶ **Simplicity:** Easy to understand and implement
- ❷ **Speed:** Fast training and prediction
- ❸ **Accuracy:** Solid classification despite assumptions

EXAM TRAP: Naïve Bayes advantages = simple, fast, and accurate!

QUESTION: In the appendix problem, what classification is made for the new instance?

OPTIONS:

- ❶ Buy (probability = 0.98)
- ❷ Sell (probability = 0.02)
- ❸ Cannot be classified
- ❹ Both equally likely

ANSWER: A

DETAILED TUTORIAL:

- **Data:**
 - 5 buys, 5 sells
 - Prior: $P(buy) = 0.5$, $P(sell) = 0.5$
 - New document: words 1, 2, 3 present; words 4, 5 absent
- **Calculations:**
 - $P(buy) \times \prod P(w_i|buy) = 0.0768$
 - $P(sell) \times \prod P(w_i|sell) = 0.00192$
- **Posterior:** $P(buy|words) = 0.98$, $P(sell|words) = 0.02$

EXAM TRAP: Work through the full calculation - **buy is much more likely!**

QUESTION: What is word embedding in NLP?

OPTIONS:

- ❶ A method to remove words
- ❷ A technique to capture word meaning by representing words in a continuous vector space
- ❸ A method to count word frequencies
- ❹ A technique to stem words

ANSWER: B

DETAILED TUTORIAL:

- **Word embedding definition:**
 - Represents words as dense vectors
 - Captures semantic meaning
 - Similar words have similar vectors

EXAM TRAP: Word embeddings = capture meaning in dense vectors!

QUESTION: What is Word2Vec?

OPTIONS:

- ❶ A word stemming algorithm
- ❷ A Google-developed algorithm for generating word embeddings using neural networks
- ❸ A dictionary approach to NLP
- ❹ A pre-processing technique

ANSWER: B

DETAILED TUTORIAL:

- **Word2Vec definition:**
 - Developed by Google
 - Neural network-based algorithm
 - Generates word embeddings
- **Two architectures:**
 - ❶ **CBOW:** Predict target from context
 - ❷ **Skip-gram:** Predict context from target

EXAM TRAP: Word2Vec = Google algorithm for word embeddings!

QUESTION: How do word embeddings capture context?

OPTIONS:

- ① By ignoring context
- ② By paying attention to positions of words and their surrounding words
- ③ By counting all words equally
- ④ By removing all punctuation

ANSWER: B

DETAILED TUTORIAL:

- Context capture methods:
 - ① **Position information:** Track which words appear together
 - ② **Surrounding words:** Consider words before and after

EXAM TRAP: Word embeddings capture context by **analyzing surrounding words!**

QUESTION: What is the difference between Skip-gram and CBOW?

OPTIONS:

- ❶ They are the same algorithm
- ❷ Skip-gram predicts context from target; CBOW predicts target from context
- ❸ Skip-gram is for English only
- ❹ CBOW is for English only

ANSWER: B

DETAILED TUTORIAL:

- **CBOW:** Context words $\rightarrow$ target word (faster, better for frequent words)
- **Skip-gram:** Target word $\rightarrow$ context words (better for rare words, more expensive)

EXAM TRAP: Skip-gram = target $\rightarrow$ context; CBOW = context $\rightarrow$ target!

QUESTION: What is one-hot encoding in NLP?

OPTIONS:

- ❶ A technique to encode temperature
- ❷ Representing words as binary vectors with a 1 in the position of that word and 0 elsewhere
- ❸ A method to count word frequencies
- ❹ A technique to remove stop words

ANSWER: B

DETAILED TUTORIAL:

- **One-hot encoding:**
 - Vector length = vocabulary size
 - One position = 1, all others = 0
 - Very sparse, no semantic meaning

EXAM TRAP: One-hot = binary vector with one 1 - no semantic meaning!

QUESTION: Why is dimensionality reduction important in NLP?

OPTIONS:

- ❶ To make documents longer
- ❷ To reduce sparse, high-dimensional vectors to dense, lower-dimensional representations
- ❸ To add more features
- ❹ To increase computation time

ANSWER: B

DETAILED TUTORIAL:

- **The problem:**
 - Vocabulary can have 10,000+ words
 - Very sparse, slow, and inefficient
- **Reduction methods:** PCA, SVD, Word2Vec

EXAM TRAP: Dimensionality reduction = **sparse** → **dense** representations!

QUESTION: What is a key advantage of word embeddings over BoW?

OPTIONS:

- ① Embeddings are sparser
- ② Embeddings capture semantic meaning and relationships between words
- ③ Embeddings are faster to compute
- ④ Embeddings don't require pre-processing

ANSWER: B

DETAILED TUTORIAL:

- Embedding advantages:
 - ① **Semantic meaning:** Similar words close in space
 - ② **Dense vectors:** Lower dimensional, more efficient
 - ③ **Context awareness:** Considers surrounding words

EXAM TRAP: Embeddings = capture semantic meaning - BoW doesn't!

QUESTION: What is Skip-gram used for in NLP?

OPTIONS:

- ❶ To remove stop words
- ❷ To predict surrounding words given a target word
- ❸ To count word frequencies
- ❹ To stem words

ANSWER: B

DETAILED TUTORIAL:

- Skip-gram architecture:
 - Input: target word
 - Output: context words
 - Uses neural network

EXAM TRAP: Skip-gram = target word → predict context words!

QUESTION: What is a famous semantic relationship captured by Word2Vec?

OPTIONS:

- ❶ "king - man + woman equals queen"
- ❷ "king + man + woman equals queen"
- ❸ "king - queen equals man - woman"
- ❹ All words are equally related

ANSWER: A

DETAILED TUTORIAL:

• **Famous analogy:**

- $v_{king} - v_{man} + v_{woman} \approx v_{queen}$
- Captures gender relationship
- Preserves arithmetic operations

EXAM TRAP: Word2Vec captures **semantic relationships** via vector arithmetic!

QUESTION: How are NLP models evaluated?

OPTIONS:

- ① Only by accuracy
- ② Using confusion matrices, accuracy, precision, recall, F1 when labels exist
- ③ Only by speed
- ④ Only by data requirements

ANSWER: B

DETAILED TUTORIAL:

• **Labeled data evaluation:**

- ① **Confusion Matrix:** TP, TN, FP, FN
- ② **Accuracy:** Overall correctness
- ③ **Precision:** Of predicted positives, how many were right?
- ④ **Recall:** Of actual positives, how many did we catch?
- ⑤ **F1:** Harmonic mean of precision and recall

EXAM TRAP: Evaluation = **confusion matrix metrics** when labels exist!

QUESTION: What metrics are used to evaluate NLP classification models?

OPTIONS:

- ❶ Only accuracy
- ❷ Accuracy, precision, recall, and F1 from confusion matrix
- ❸ Only precision
- ❹ Only recall

ANSWER: B

DETAILED TUTORIAL:

• **Standard evaluation metrics:**

- ❶ **Accuracy:** $(TP + TN) / (TP + TN + FP + FN)$
- ❷ **Precision:** $TP / (TP + FP)$
- ❸ **Recall:** $TP / (TP + FN)$
- ❹ **F1:** Harmonic mean of precision and recall

EXAM TRAP: Use **multiple metrics** - not just accuracy!

QUESTION: How is NLP evaluation different for unlabeled data?

OPTIONS:

- ❶ No evaluation is possible
- ❷ Use unsupervised evaluation methods or qualitative assessment
- ❸ Use the same metrics as labeled data
- ❹ Evaluation is not needed

ANSWER: B

DETAILED TUTORIAL:

- **Unlabeled data evaluation:**
 - Subject matter expert review
 - Unsupervised metrics (cohesion, separation)
 - Indirect validation

EXAM TRAP: Unlabeled data requires **alternative evaluation methods!**

QUESTION: Why should speed and data requirements be considered in NLP evaluation?

OPTIONS:

- ① They are not important
- ② Textual databases can be vast and sparse vectors are hard to handle efficiently
- ③ Speed is never a concern
- ④ All NLP models are equally fast

ANSWER: B

DETAILED TUTORIAL:

• **Practical considerations:**

- ① **Data volume:** NLP datasets can be enormous
- ② **Sparsity:** Sparse vectors are inefficient
- ③ **Model complexity:** Neural networks are slow to train

EXAM TRAP: NLP can be **computationally expensive** - speed and data matter!

QUESTION: What is a major limitation of simple BoW methods?

OPTIONS:

- ① They are too fast
- ② They are not useful for determining context or intended meaning
- ③ They are too accurate
- ④ They require too much data

ANSWER: B

DETAILED TUTORIAL:

• **BoW limitations:**

- ① **Loss of context:** Word order ignored
- ② **No semantic meaning:** No concept of similarity
- ③ **Negation issues:** "not good" vs "good"

EXAM TRAP: BoW fails to capture **context and meaning**!

QUESTION: What advanced techniques help determine word context?

OPTIONS:

- ➊ Only BoW
- ➋ n-grams, Skip-grams, and word embeddings
- ➌ Only stop word removal
- ➍ Only stemming

ANSWER: B

DETAILED TUTORIAL:

- Context-aware techniques:
 - ➊ **n-grams:** n contiguous words as one unit
 - ➋ **Skip-grams:** Predict context from target
 - ➌ **Word embeddings:** Dense vector representations

EXAM TRAP: n-grams, Skip-grams, and embeddings capture context!

QUESTION: What is the advantage of TF-IDF over simple word counts?

OPTIONS:

- ① TF-IDF is faster to compute
- ② TF-IDF assigns higher weights to rare but important words
- ③ TF-IDF counts all words equally
- ④ TF-IDF doesn't require a corpus

ANSWER: B

DETAILED TUTORIAL:

- TF-IDF advantages:
 - ① **Weights rare words higher:** More discriminatory power
 - ② **Weights common words lower:** Reduced noise

EXAM TRAP: TF-IDF = higher weight for rare words!

QUESTION: How is document similarity measured in NLP?

OPTIONS:

- ❶ By comparing document titles
- ❷ By comparing document vectors using similarity metrics (cosine similarity)
- ❸ By counting words in each document
- ❹ By comparing document lengths

ANSWER: B

DETAILED TUTORIAL:

- **Similarity measurement:**
 - ❶ **Document vectors:** Using BoW, TF-IDF, or embeddings
 - ❷ **Similarity metrics:** Cosine similarity, Jaccard, Euclidean

EXAM TRAP: Document similarity = **comparing document vectors** - often cosine similarity!

QUESTION: What is cosine similarity used for in NLP?

OPTIONS:

- ❶ To count word frequencies
- ❷ To measure similarity between two document vectors
- ❸ To remove stop words
- ❹ To stem words

ANSWER: B

DETAILED TUTORIAL:

• **Cosine similarity formula:**

- $\cos(\theta) = \frac{v_1 \cdot v_2}{||v_1|| \times ||v_2||}$
- Range: -1 to 1
- cos equals 1 = very similar

EXAM TRAP: Cosine similarity = [measures document vector similarity](#)!

QUESTION: What is the tradeoff between stemming and lemmatization?

OPTIONS:

- ❶ No tradeoff - they are the same
- ❷ Stemming is faster but less accurate; lemmatization is slower but more accurate
- ❸ Stemming is more accurate
- ❹ Lemmatization is faster

ANSWER: B

DETAILED TUTORIAL:

- ❶ **Stemming:** Fast, less accurate, may create non-words
- ❷ **Lemmatization:** Slow, more accurate, always produces real words

EXAM TRAP: Stemming = fast but less accurate; Lemmatization = slow but more accurate!

QUESTION: What are the main challenges in NLP?

OPTIONS:

- ❶ Only computational speed
- ❷ Ambiguity, context dependence, noise, and sarcasm/detecting tone
- ❸ Only data size
- ❹ Only language barriers

ANSWER: B

DETAILED TUTORIAL:

• **Key challenges:**

- ❶ **Ambiguity:** Same words have different meanings
- ❷ **Context dependence:** Meaning depends on surrounding words
- ❸ **Noise:** Spelling errors, abbreviations, colloquialisms
- ❹ **Tone detection:** Sarcasm, sentiment nuances

EXAM TRAP: NLP challenges include **ambiguity, context, noise, and tone!**

QUESTION: What are the basic steps for sentiment analysis using a dictionary approach?

OPTIONS:

- ❶ Only count positive words
- ❷ Establish dictionary, pre-process text, replace with stems/lemmas, calculate proportions
- ❸ Only count negative words
- ❹ Use machine learning only

ANSWER: B

DETAILED TUTORIAL:

• **Sentiment analysis steps:**

- ❶ **Establish dictionary:** Positive, negative, neutral lists
- ❷ **Pre-process text:** Remove punctuation, stop words, lowercase
- ❸ **Replace with stems/lemmas:** Standardize word forms
- ❹ **Calculate proportions:** Positive, negative, neutral

EXAM TRAP: Steps = dictionary, pre-process, stem, calculate proportions!

QUESTION: Explain how a bag of words model is used in NLP.

OPTIONS:

- ❶ It preserves word order
- ❷ It treats each term identically, counts frequencies, stores in vectors for analysis
- ❸ It removes all words
- ❹ It only works for numbers

ANSWER: B

DETAILED TUTORIAL:

• **BoW process:**

- ❶ Treat terms identically (ignore order)
- ❷ Count frequencies of each word
- ❸ Store in vectors

EXAM TRAP: BoW = **treats** each word **identically**, **counts**, **stores in vectors**!

QUESTION: What are the two assumptions of Naïve Bayes for document grouping?

OPTIONS:

- ❶ Words are ordered and have different weights
- ❷ Each word is independent of others and each word has equal weighting
- ❸ Words are dependent and have different weights
- ❹ Words are random

ANSWER: B

DETAILED TUTORIAL:

- ❶ **Assumption 1: Independence**
 - $P(w_1, w_2, \dots, w_n|c) = P(w_1|c) \times P(w_2|c) \times \dots$
- ❷ **Assumption 2: Equal weighting**
 - Each word has equal information value

EXAM TRAP: Naïve Bayes = **independence** + **equal weighting** - two key assumptions!

QUESTION: Why do negation words cause problems and how can they be handled?

OPTIONS:

- ❶ They don't cause problems
- ❷ Negations reverse meaning; use n-grams to pair negation with following words
- ❸ Remove all negation words
- ❹ Treat negation as positive

ANSWER: B

DETAILED TUTORIAL:

- **The negation problem:**
 - "not adequate" vs "adequate"
 - BoW sees "adequate" in both
 - Meaning is opposite
- **n-gram solution:**
 - Treat "not adequate" as one unit
 - Bi-grams capture negation

EXAM TRAP: Negation problem = solved with n-grams pairing negation + following word!

QUESTION: Why do some words have zero TF-IDF values?

OPTIONS:

- ① Because they are misspelled
- ② Because they appear in every document and have no discriminatory power
- ③ Because they are too long
- ④ Because they are not in the vocabulary

ANSWER: B

DETAILED TUTORIAL:

- **TF-IDF = 0 condition:**
 - Word appears in all documents
 - $IDF = \ln(D/D) = \ln(1) = 0$
 - $TF-IDF = TF \times 0 = 0$

EXAM TRAP: Zero TF-IDF = word appears in all documents - no discriminatory power!

QUESTION: Why does the same word have different TF-IDF scores in different documents?

OPTIONS:

- ❶ Because IDF varies by document
- ❷ Because TF varies by document (document length and frequency differences)
- ❸ Because vocabulary changes
- ❹ Because documents have different authors

ANSWER: B

DETAILED TUTORIAL:

- **TF variation:**
 - $TF = w_{i,j} / |v_j|$
 - Different lengths $\Rightarrow$ different TFs
- **IDF is constant:** Depends only on corpus

EXAM TRAP: TF varies by document $\Rightarrow$ TF-IDF varies by document!

QUESTION: If new words appear in test documents not in the training sample, how should they be treated?

OPTIONS:

- ❶ Add them to the vocabulary
- ❷ Ignore them because they cannot help with classification
- ❸ Remove the entire document
- ❹ Use them as stop words

ANSWER: B

DETAILED TUTORIAL:

- **Unknown word handling:**
 - Words not in training vocabulary
 - No information about their distribution
 - Cannot calculate $P(w|c)$
 - Must be ignored

EXAM TRAP: Unknown words = **ignored** - no information about them!

QUESTION: Why is smoothing required in Naïve Bayes applications and how does it work?

OPTIONS:

- ❶ It's not required
- ❷ To avoid zero probability problems by adding a positive integer to all counts
- ❸ To remove stop words
- ❹ To stem words

ANSWER: B

DETAILED TUTORIAL:

- **Why smoothing is needed:**
 - Zero probability problem
 - Word not in training class $\rightarrow P = 0$
 - Product becomes zero
- **How smoothing works:**
 - Add λ to all counts
 - $\lambda = 1$: Laplace smoothing

EXAM TRAP: Smoothing = add to counts to avoid zero probabilities!

QUESTION: What happens to the probabilities under Laplace smoothing?

OPTIONS:

- ❶ They become more extreme
- ❷ They are pushed toward a uniform distribution
- ❸ They become negative
- ❹ They don't change

ANSWER: B

DETAILED TUTORIAL:

- **Effect of Laplace smoothing:**
 - Adds counts to all classes
 - All probabilities become positive
 - Shrinks toward uniform distribution
 - Similar to regularization

EXAM TRAP: Laplace smoothing = **shrinks** toward uniform distribution!

QUESTION: In the appendix problem, what is the most likely recommendation?

OPTIONS:

- ① Buy (probability = 0.98)
- ② Sell (probability = 0.02)
- ③ Hold
- ④ Cannot be determined

ANSWER: A

DETAILED TUTORIAL:

- **Given data:**
 - 5 buys, 5 sells (prior = 0.5 each)
 - New document: words 1-3 present, words 4-5 absent
- **Calculations:**
 - Buy: $0.8 \times 1.0 \times 0.4 \times 0.8 \times 0.6 \times 0.5 = 0.0768$
 - Sell: $0.2 \times 0.4 \times 0.6 \times 0.4 \times 0.2 \times 0.5 = 0.00192$
- **Result:** Buy (98% probability)

EXAM TRAP: Work through the full calculation - **buy is much more likely!**

QUESTION: What are the main pre-processing steps in NLP?

OPTIONS:

- ❶ Only tokenization
- ❷ Tokenization, stop word removal, stemming/lemmatization, and feature extraction
- ❸ Only stemming
- ❹ Only stop word removal

ANSWER: B

DETAILED TUTORIAL:

• Pre-processing steps:

- ❶ **Tokenization:** Split into sentences and words
- ❷ **Stop word removal:** Remove common words with no value
- ❸ **Stemming/Lemmatization:** Reduce to root form
- ❹ **Feature extraction:** Convert to numeric vectors

EXAM TRAP: Pre-processing = tokenization, stop words, stemming/lemmatization, feature extraction!

QUESTION: What is the difference between sentence tokenization and word tokenization?

OPTIONS:

- ❶ They are the same thing
- ❷ Sentence tokenization splits at punctuation; word tokenization splits into words
- ❸ Sentence tokenization removes words
- ❹ Word tokenization removes sentences

ANSWER: B

DETAILED TUTORIAL:

- **Sentence tokenization:** Splits at periods, question marks, exclamation marks
- **Word tokenization:** Splits sentences into words, removes punctuation

EXAM TRAP: Sentence = **splitting at punctuation**; Word = **splitting into words**!

QUESTION: Why do stop words vary across applications?

OPTIONS:

- ❶ All stop words are the same
- ❷ Words with no value in one context may be useful in another
- ❸ Stop words are always removed
- ❹ Stop words never matter

ANSWER: B

DETAILED TUTORIAL:

- **Context-dependent stop words:**
 - Some words may be useful in specific domains
 - "bank" is common but important in financial text
 - Standard lists may remove important words

EXAM TRAP: Stop words are **context-dependent** - not one-size-fits-all!

QUESTION: In Figure 9.1, what does the Document Term Matrix show?

OPTIONS:

- ❶ Document titles and authors
- ❷ Rows as documents, columns as words, entries as word counts
- ❸ Document dates
- ❹ Document sources

ANSWER: B

DETAILED TUTORIAL:

- **DTM structure:**
 - **Rows:** Each row = one document
 - **Columns:** Each column = one word from vocabulary
 - **Entries:** Word counts or frequencies

EXAM TRAP: DTM = documents $\times$ words matrix with counts!

QUESTION: Why is feature extraction (text representation) important in NLP?

OPTIONS:

- ❶ To make text longer
- ❷ To convert text into numeric vectors for machine learning analysis
- ❸ To remove words
- ❹ To make text more readable

ANSWER: B

DETAILED TUTORIAL:

- **Feature extraction purpose:**
 - Machine learning models need numeric inputs
 - Text must be converted to numbers
 - Enables statistical analysis

EXAM TRAP: Feature extraction = text → numeric vectors!

QUESTION: What are the dimensions of a Document Term Matrix?

OPTIONS:

- ❶ $D \times W$ (documents $\times$ words)
- ❷ $W \times D$ (words $\times$ documents)
- ❸ $D \times D$ (documents $\times$ documents)
- ❹ $W \times W$ (words $\times$ words)

ANSWER: A

DETAILED TUTORIAL:

- **DTM dimensions:**
 - Rows: D = number of documents
 - Columns: W = number of words in vocabulary
 - Dimensions: $D \times W$

EXAM TRAP: DTM dimensions = documents $\times$ words!

QUESTION: In the BoW example, why does d1 have a 2 for "high" in the count vector?

OPTIONS:

- ❶ Because "high" is the most important word
- ❷ Because "high" appears twice in the document
- ❸ Because "high" is a stop word
- ❹ Because "high" is the first word

ANSWER: B

DETAILED TUTORIAL:

- **Example text:**
 - d1: "... reaching a high of 1550 ... with a high of 1560"
 - "high" appears twice
 - Count vector records 2

EXAM TRAP: Count BoW = **actual frequency** - not just presence!

QUESTION: What is the Bag of n-grams (BoN) approach?

OPTIONS:

- ➊ A method to count single words
- ➋ Applying BoW to n-grams (n contiguous words as units)
- ➌ A method to remove n-grams
- ➍ A method to count only bigrams

ANSWER: B

DETAILED TUTORIAL:

- **BoN definition:**
 - Extends BoW to n-grams
 - n contiguous words = one unit
 - Handles phrases and negations

EXAM TRAP: BoN = BoW applied to n-grams - captures phrases!

QUESTION: In document similarity, what does a cosine similarity close to 1 indicate?

OPTIONS:

- ❶ Documents are very different
- ❷ Documents are very similar (vectors point in same direction)
- ❸ Documents are opposite
- ❹ Documents are unrelated

ANSWER: B

DETAILED TUTORIAL:

- **Cosine similarity interpretation:**
 - $\cos$ equals 1: Very similar documents
 - $\cos$ equals 0: Unrelated documents
 - $\cos$ equals -1: Opposite documents

EXAM TRAP: $\cos$ equals 1 = very similar documents!

QUESTION: What are the key takeaways from the NLP chapter?

OPTIONS:

- ❶ Only BoW is used in NLP
- ❷ NLP involves pre-processing, feature extraction, dictionary or ML approaches, and careful evaluation
- ❸ Only machine learning is used
- ❹ NLP doesn't require pre-processing

ANSWER: B

DETAILED TUTORIAL:

• **Key takeaways:**

- ❶ **Pre-processing:** Tokenization, stop words, stemming/lemmatization
- ❷ **Feature extraction:** BoW, n-grams, TF-IDF
- ❸ **Approaches:** Dictionary-based and machine learning
- ❹ **Evaluation:** Confusion matrix metrics, speed, data requirements

EXAM TRAP: NLP = pre-processing + feature extraction + modeling + evaluation!

Summary: Complete NLP Review

Pre-processing:

- **Tokenization:** Sentences $\rightarrow$ words
- **Stop Words:** Remove common words with no value
- **Stemming:** Cut prefixes/suffixes (fast, less accurate)
- **Lemmatization:** Dictionary form (slow, more accurate)

Feature Extraction:

- **BoW:** Binary (presence) or Count (frequency)
- **n-grams:** Contiguous word sequences
- **TF-IDF:** $TF \times IDF$ - weights rare words higher
- **Embeddings:** Dense vectors capturing meaning (Word2Vec)

Classification:

- **Dictionary:** Pre-defined word lists (sentiment analysis)
- **Naïve Bayes:** Simple, fast, generative classifier
- **Smoothing:** Laplace smoothing to avoid zero probabilities

Evaluation:

- **Confusion Matrix:** TP, TN, FP, FN
- **Metrics:** Accuracy, Precision, Recall, F1
- **Challenges:** Ambiguity, context, negation, sarcasm

Exam Success: Understand the process from raw text to classification and evaluation!